

Discovery Agent

Enterprise System Discovery & Integration Intelligence Platform

Evidence-backed system discovery, integration gap analysis, and safe connector generation from enterprise documents.

Project positioning

A production-oriented engineering portfolio project that combines document intelligence, deterministic reasoning, optional LLM enrichment, integration architecture, secure code generation, a professional React operator console, containerized validation, and deployment hardening.

Prepared by	Mathan M.
Project type	Full-stack AI / Enterprise Integration Engineering
Primary stack	Python 3.12 · FastAPI · SQLAlchemy · React 19 · TypeScript · Docker
Document version	1.0
Report date	September 2026

CHAPTER 00

Document Purpose

A self-contained technical reference for project review, deployment, interview preparation, and future development.

This report explains Discovery Agent from the problem it solves through its internal architecture, algorithms, security boundaries, user interface, deployment model, testing strategy, and future roadmap. It is written so that a reviewer can understand the project without reading the source code first.

Current engineering status

The core local workflow is implemented and has previously passed the full backend/frontend quality gates. A new deployment-hardening layer (workspace isolation, rate limiting, PostgreSQL compatibility, privacy-mode storage and static hosted validation) is currently being integrated; API regression tests are being updated for the new session boundary before public release.

CHAPTER 00

Table of Contents

01 Abstract	4	16 Deployment Architecture	19
02 Introduction and Motivation	5	17 API Reference	20
03 Objectives, Scope and Success Criteria	6	18 Configuration Reference	21
04 Solution Overview	7	19 Testing, CI and Quality Engineering	22
05 End-to-End Workflow	8	20 Worked Example: Invoice Automation	23
06 Backend Architecture	9	21 Professionalism and Unique Design Choices	24
07 Document Ingestion and Privacy Pipeline	10	22 Limitations and Risk Register	25
08 Evidence-Backed System Discovery	11	23 Roadmap	26
09 AI Provider Routing and Failure Tolerance	12	24 Local Development and Operations	27
10 Persistence and Data Model	13	25 Deployment Checklist	28
11 Use Cases and Gap Analysis	14	26 Viva and Interview Questions	29
12 Connector and Agent Generation	15	27 Glossary	30
13 Generated-Code Validation	16	28 Repository Map and File Responsibilities	31
14 Frontend Operator Console	17	29 Conclusion	32
15 Security Architecture	18	30 Appendix A - Suggested Production Environment	33

Page numbers reflect this report edition.

CHAPTER 01

Abstract

Discovery Agent is a full-stack enterprise system-discovery and integration-planning platform that converts unstructured business and architecture documents into structured integration intelligence. The platform ingests common enterprise file formats, performs limited sensitive-data redaction, discovers software systems with source evidence, builds a normalized application inventory, captures automation use cases, performs integration gap analysis, generates Python or Node.js connector scaffolds, and validates generated artifacts using a secure validation strategy.

The system deliberately combines deterministic logic with optional large-language-model enrichment. This hybrid design keeps the application functional when external AI providers are unavailable or rate-limited, while still benefiting from model-assisted extraction and generation when available. Groq and Gemini are routed through a provider-neutral failover layer, followed by deterministic fallback behavior.

A central design goal is trust. Discovery findings retain evidence and provenance; inferred items receive lower confidence and human-review flags; LLM-discovered systems are checked against source text; public deployment can isolate anonymous user workspaces; rate limits protect free-tier AI quotas; and generated code is never silently executed on a public host. The project therefore demonstrates practical AI engineering together with conventional software architecture, security, DevOps and enterprise integration principles.

CHAPTER 02

Introduction and Motivation

Business problem

Before an organization automates a workflow, architects must understand the existing system estate: what platforms are present, what they do, how they authenticate, what data they contain, which business processes depend on them, and which required integrations are missing. In many real organizations this information is fragmented across PDFs, spreadsheets, architecture diagrams, process documents, support notes and tribal knowledge.

Why the problem is difficult

- Enterprise documentation is heterogeneous and frequently outdated.
- System names may be explicit, abbreviated, inferred from processes, or duplicated across documents.
- A purely LLM-based extractor can hallucinate systems or overstate confidence.
- A purely deterministic matcher is predictable but cannot interpret every novel description.
- Integration planning requires reasoning over available systems, missing capabilities, business criticality and data flows.
- Generated integration code is dangerous if executed directly on a host without isolation.
- A public demo must protect API quotas, user data and shared state.

Project idea

Discovery Agent addresses these problems as an end-to-end operator workflow rather than a single model prompt. The user uploads enterprise documentation, reviews evidence-backed discoveries, defines or auto-discovers an automation goal, receives an integration gap analysis, generates a connector scaffold for missing capability, validates the generated artifact, and exports structured reports.

CHAPTER 03

Objectives, Scope and Success Criteria

Primary objectives

1. Ingest common enterprise document formats with defensible parsing behavior.
2. Discover software systems while retaining source evidence and confidence metadata.
3. Reduce hallucination risk using deterministic matching and source-grounding checks.
4. Build a structured application inventory that can be searched and reviewed.
5. Map business automation goals to available and missing systems.
6. Produce integration gap reports with dependencies, data flows, effort and priority indicators.
7. Generate connector scaffolds in Python and Node.js.
8. Validate generated code safely without making public deployment depend on host execution.
9. Provide a professional operator interface suitable for demonstration and real evaluation.
10. Support local development and a low-cost/free hosted portfolio deployment path.

Non-goals

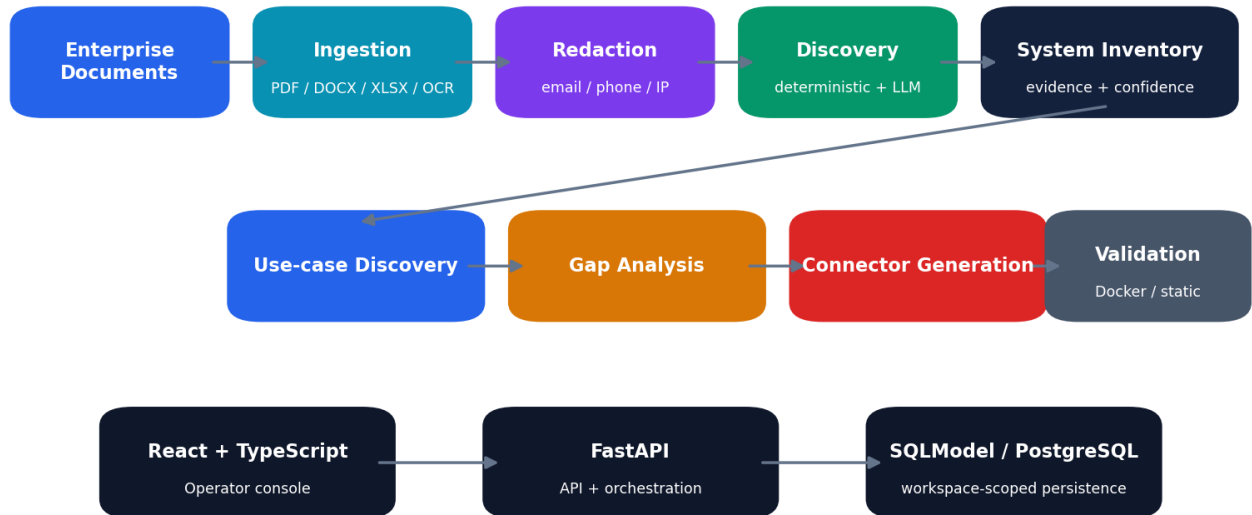
- The generated connector is not claimed to be a vendor-certified production integration.
- Confidence scores are implementation heuristics, not statistically calibrated probabilities.
- The redactor is pattern-based and not a complete enterprise DLP/PII solution.
- The free public deployment is a portfolio/demo target, not a high-availability commercial SLA.

CHAPTER 04

Solution Overview

Discovery Agent - System Architecture

Evidence-first enterprise discovery, gap analysis and connector generation



Optional model layer: Groq → Gemini → deterministic fallback

Figure 4.1 - High-level Discovery Agent architecture

The platform is organized around an evidence-first pipeline. The React frontend acts as an operator console; FastAPI orchestrates document ingestion, discovery, persistence, gap analysis, code generation and reporting; SQLModel defines domain models; optional LLM providers enrich reasoning; and validation behavior changes according to environment.

Stage	Input	Primary output	Trust mechanism
Ingestion	Enterprise file	Text chunks + metadata	format-specific parsing, bounded upload
Redaction	Extracted text	Sanitized chunks	email/phone/IP patterns
Discovery	Chunks	System findings	catalog matching + LLM grounding
Inventory	Findings	Normalized systems	deduplication + evidence retention
Gap analysis	Use case + inventory	Available/missing systems	deterministic fallback + AI reasoning
Generation	Gap entry	Connector package	templates + optional LLM
Validation	Generated code	Pass/fail + logs	Docker locally; static-only hosted
Reporting	Persisted records	JSON/CSV/UI reports	workspace scoping

CHAPTER 05

End-to-End Workflow

End-to-End Processing Flow



Trust controls applied throughout

- **Source provenance** Evidence sentence, source file, page/line
- **Hallucination guard** Reject model systems not supported by source text
- **Workspace isolation** Signed anonymous workspace cookie for hosted mode
- **Rate control** Per-workspace quotas for expensive operations
- **Fail-closed validation** No silent host execution in production

Figure 5.1 - Processing pipeline and trust controls

Typical user journey

11. Upload a PDF, DOCX, XLSX, CSV, text, Markdown or supported image.
12. The backend validates the filename, extension and size before streaming the file to storage.
13. The ingestor extracts structured text and metadata; the redactor masks supported sensitive patterns.
14. The discovery engine produces systems with category, authentication method, key entities, business processes, criticality, confidence, evidence and provenance.
15. The user reviews the inventory and creates an automation use case or asks the system to discover candidate goals.
16. The mapping engine compares required systems to inventory and produces gap, dependency, data-flow, impact and priority analysis.
17. For a missing integration, the connector generator produces code, tests, dependencies, configuration files, a README and an agent definition.
18. Validation runs according to environment: Docker-isolated locally or syntax/static-only in public hosted mode.
19. Reports and validation history are persisted and displayed in the operator console.

CHAPTER 06

Backend Architecture

FastAPI orchestration

backend/main.py is the application composition layer. It configures startup validation, middleware, CORS, host restrictions, database sessions, upload handling, inventory persistence, use-case operations, gap analysis, artifact generation, validation and reporting endpoints.

Core modules

Module	Responsibility
core/ingestor.py	Multi-format extraction and chunk construction
core/redactor.py	Pattern-based masking of email, phone and IPv4 data
core/discovery_catalog.py	Deterministic catalog of known enterprise systems and aliases
core/extractor.py	Evidence-backed system discovery and LLM result normalization
core/provider_router.py	Groq/Gemini provider order and failover
core/llm.py	Groq integration and JSON parsing
core/gemini_provider.py	Gemini SDK integration and JSON parsing
core/mapping_engine.py	Use-case to inventory mapping, gaps, data flow and priority
core/code_gen.py	Connector and agent-definition generation
core/code_generation_provider.py	Provider-neutral hook into generator
core/sandbox.py	Static/local/Docker validation strategies
core/security.py	Signed workspaces, rate limits and security headers

Configuration philosophy

Runtime behavior is controlled with environment variables under the DISCOVERY_ prefix. Development defaults remain convenient, while production validation explicitly rejects unsafe combinations such as SQLite persistence or Docker/local generated-code execution on a public host.

CHAPTER 07

Document Ingestion and Privacy Pipeline

Supported content

- PDF - text extraction using pdfplumber.
- DOCX - document text extraction.
- XLSX/CSV - structured spreadsheet content.
- TXT/Markdown - direct text.
- Images - OCR path using Pillow and Tesseract where available.

Upload controls

- Filename normalization prevents path traversal.
- Extension allow-listing rejects unsupported files.
- Streaming upload avoids loading the entire object into memory at once.
- Maximum file size is configurable.
- Partial files are removed if upload fails.
- Duplicate names receive a unique suffix.

Hosted privacy mode

The deployment-hardening layer adds a privacy-oriented option to persist redacted extracted text while deleting the original uploaded file after processing. This addresses the ephemeral filesystem limitation of free hosting and reduces retention of potentially sensitive source documents. The retained text size is bounded by configuration.

Important limitation

Current redaction masks email addresses, phone numbers and IPv4 addresses. It does not provide comprehensive named-entity de-identification and should not be represented as full PII anonymization.

CHAPTER 08

Evidence-Backed System Discovery

Deterministic discovery

A curated enterprise-system catalog supplies canonical names, aliases, categories, common authentication methods, entities, processes and criticality defaults. When an alias is explicitly present in text, the system creates a high-confidence finding tied to the source sentence.

Generic discovery

Generic terms such as “mainframe”, “database”, “ERP system”, “CRM tool”, “legacy app” and “cloud storage” can produce lower-confidence findings marked for human review because the exact product identity is not known.

LLM enrichment with hallucination control

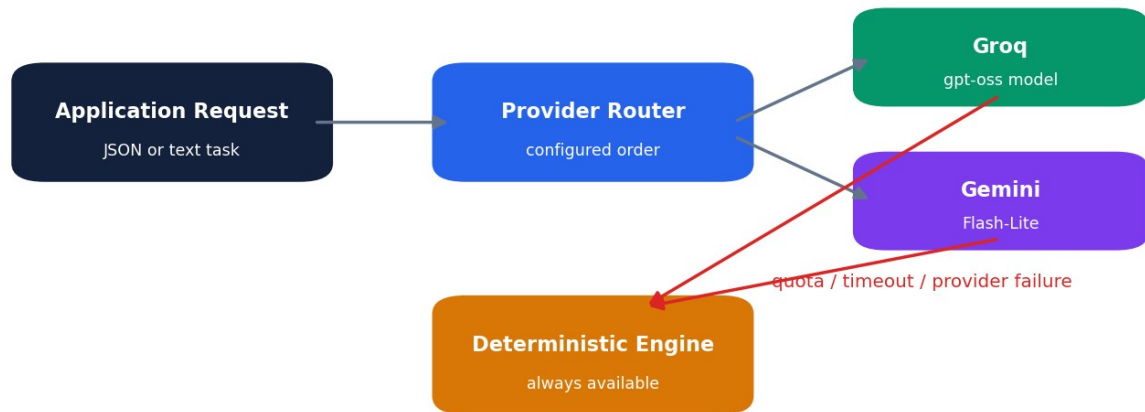
The LLM is prompted to return structured JSON findings, but model output is not blindly trusted. Each returned system name must be supported by the current source chunk. Unsupported model findings are rejected. This design makes evidence a gating condition rather than only an explanatory decoration.

Confidence zone	Interpretation	UI behavior
0.95+	Explicit system evidence	Evidence found
0.70–0.90	Contextual or inferred finding	Review context
<0.70	Uncertain finding	Human review required

CHAPTER 09

AI Provider Routing and Failure Tolerance

Provider-Neutral AI Routing



Design principle: improves quality when available, but does not become a single point of failure.

Figure 9.1 - Groq/Gemini failover with deterministic fallback

Why provider-neutral routing matters

Free-tier AI APIs can hit rate limits, models can be temporarily unavailable, and pricing or model IDs can change. Provider routing isolates this volatility from the core application. The configured order is evaluated, unavailable providers are skipped, failed requests fall through to the next provider, and deterministic logic remains the final fallback.

Current providers

Provider	Role	Configuration
Groq	Primary/secondary JSON reasoning and generation	DISCOVERY_GROQ_API_KEY + model
Gemini	Primary/secondary fallback provider	Google SDK environment configuration + DISCOVERY_GEMINI_MODEL
Deterministic engine	No-key fallback	always available

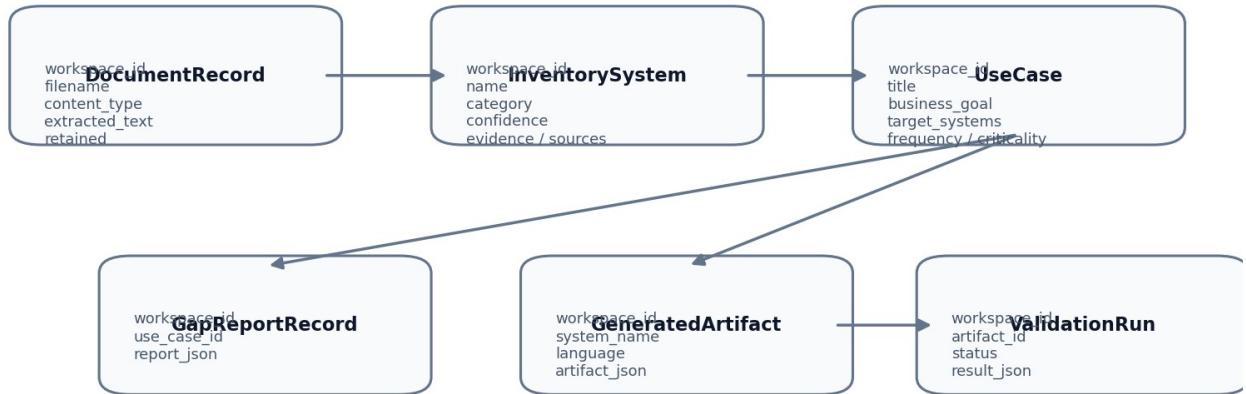
Provider status endpoint

The health/runtime response exposes only safe metadata such as provider name, configured state, selected model and failover order. API keys are never returned to the browser.

CHAPTER 10

Persistence and Data Model

Logical Data Model



Every persisted domain object can be scoped by workspace_id in hosted multi-user mode.

Figure 10.1 - Logical domain model

SQLModel layer

SQLModel provides typed domain models on top of SQLAlchemy. Development uses SQLite by default. Deployment hardening replaces SQLite-specific JSON columns with generic SQLAlchemy JSON so the same models can run against managed PostgreSQL.

Workspace scoping

Each major persisted entity now contains workspace_id. In hosted mode, the backend signs an anonymous workspace identifier in an HttpOnly cookie and filters queries by that workspace. This prevents unrelated public users from sharing the same inventory, use cases, generated artifacts and reports.

Main entities

Entity	Purpose
DocumentRecord	Upload metadata, redacted extracted text and retention state
InventorySystem	Normalized system details, confidence and provenance
UseCase	Automation goal, target systems, frequency and criticality
GapReportRecord	Persisted structured gap-analysis output
GeneratedArtifact	Connector package and metadata
ValidationRun	Validation status, logs and warnings

CHAPTER 11

Use Cases and Gap Analysis

Use-case model

A use case captures title, description, business goal, target systems, data flows, execution frequency and business criticality. The user can define it manually or invoke automatic goal discovery from uploaded documentation.

Mapping logic

The mapping engine first attempts provider-assisted reasoning, then falls back to deterministic inference. Required systems are matched against inventory names. Each system becomes available or missing; missing systems create dependencies and receive effort/priority signals. Data-flow edges are marked blocked when either endpoint is absent.

Business impact heuristic

Frequency and criticality weights produce an impact-oriented score. Missing high-criticality systems used daily become higher-priority integration gaps than low-frequency administrative gaps. This is a transparent heuristic designed for prioritization, not financial ROI estimation.

CHAPTER 12

Connector and Agent Generation

Generated package

- Implementation scaffold in Python or Node.js.
- Companion tests.
- Dependency metadata.
- README/documentation.
- Configuration files when applicable.
- Agent definition containing tools, workflow steps and test scenarios.

Design philosophy

The fallback generator intentionally produces integration scaffolding rather than pretending to know every vendor endpoint or authentication contract. A production team must still verify vendor API contracts, secrets handling, authentication flows, error semantics, idempotency and operational monitoring.

Why this is professional

The system treats generated code as a starting artifact with validation and review gates, not as automatically trusted production code.

CHAPTER 13

Generated-Code Validation

Three validation modes

Mode	Where used	Behavior	Risk profile
docker	Local/controlled environment	syntax + tests/runtime in constrained container	strong isolation when Docker available
static	Public free hosting	syntax/static checks only; no runtime execution	safe for shared hosted demo
local	Trusted developer machine only	executes generated code on host	explicitly unsafe for public use

Docker controls

- Network disabled with `--network none`.
- Memory, CPU and PID limits.
- Read-only root filesystem.
- Linux capabilities dropped.
- `no-new-privileges` security option.
- Bounded tmpfs and command timeout.
- Temporary workspace lifecycle and normalized filenames.

Fail-closed behavior

If Docker validation is requested but Docker is unavailable, the backend returns a failed validation result. It does not silently switch to host execution. Public production configuration is explicitly required to use static validation.

CHAPTER 14

Frontend Operator Console

Design goals

- Look like an internal enterprise operations product rather than an AI-generated landing page.
- Keep status and evidence visible.
- Separate workflow stages into clear pages.
- Avoid synthetic progress states and fabricated timestamps.
- Remain usable on desktop, tablet and mobile widths.

Main pages

Page	Purpose
Dashboard	workspace metrics, recent systems, pipeline and runtime state
Documents	drag/drop upload and source history
System Inventory	search, category filters, confidence, evidence and exports
Gap Analysis	use-case form, saved use cases, gap/dependency/data-flow results
Code Generation	connector configuration, generated code/tests/README and validation
Reports	consolidated operational history

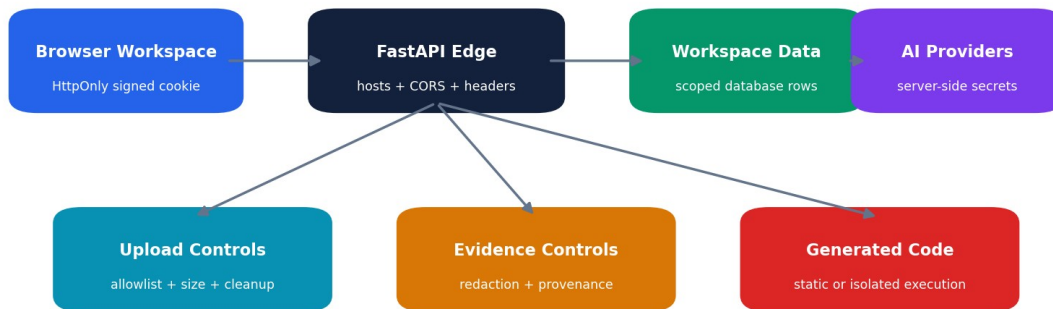
Frontend stack

React 19 and TypeScript provide the application shell and typed API contracts; Vite handles local development and production builds. Purpose-built CSS provides the visual system, removing earlier template/Tailwind residue. Vitest and Testing Library cover UI behavior.

CHAPTER 15

Security Architecture

Security and Trust Boundaries



Production rules

- PostgreSQL required - SQLite rejected
- Static validation required on public free hosting
- Session signing key required when workspace isolation is enabled
- Security headers include CSP, HSTS, frame denial and no-sniff
- Rate limits protect LLM quota and destructive endpoints

Figure 15.1 - Security and trust boundaries

Anonymous workspace isolation

Hosted mode can issue a random 32-character workspace ID signed with HMAC-SHA256. The cookie is HttpOnly, Secure in production and SameSite=Lax. Because the server signs the identifier, a client cannot forge another workspace by editing the ID alone.

Rate limiting

A lightweight in-memory limiter protects expensive and destructive endpoints per workspace and client address. Example quotas include six document uploads per hour, ten connector generations per hour and twenty provider-verification requests per hour. For multi-instance production, this would migrate to a shared store such as Redis.

HTTP response hardening

- X-Content-Type-Options: nosniff
- X-Frame-Options: DENY
- Referrer-Policy: no-referrer
- Permissions-Policy disables camera, microphone and geolocation
- Cross-Origin-Opener-Policy: same-origin
- Production CSP and HSTS
- Per-request correlation ID returned as X-Request-ID

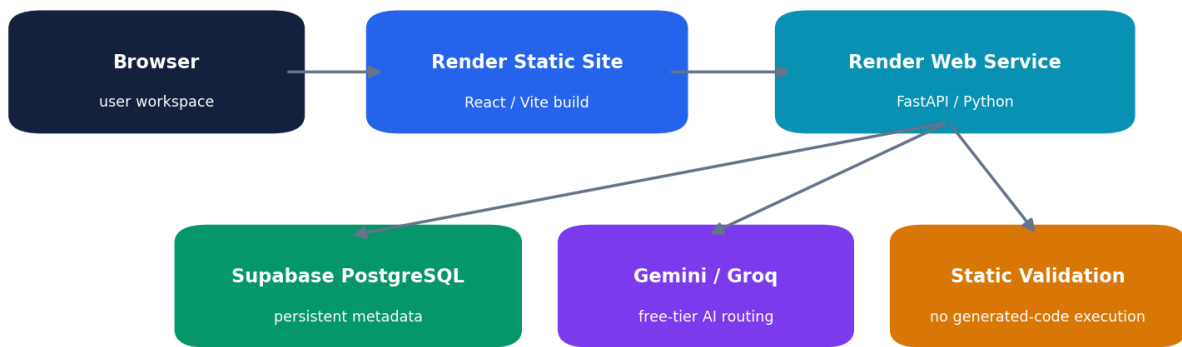
Secrets

Model keys, database credentials and session secrets belong only in server-side environment variables or a secret manager. The .env file is excluded from version control, frontend VITE_* variables must never contain secrets, and API responses expose only safe provider metadata.

CHAPTER 16

Deployment Architecture

Recommended Free Portfolio Deployment



Hosted-demo privacy mode: retain redacted extracted text; optionally delete original uploads after processing.

Figure 16.1 - Free portfolio deployment topology

Recommended free stack

Layer	Recommended service	Reason
Frontend	Render Static Site	free HTTPS static hosting
API	Render Free Web Service	free FastAPI hosting for demo workloads
Database	Supabase PostgreSQL Free	persistent SQL storage compatible with SQLAlchemy
AI	Gemini + Groq free tiers	provider redundancy
Generated code validation	Static mode on host	no execution of untrusted generated code

Why not MongoDB for this project

The current application is SQLAlchemy based. Switching to MongoDB would require replacing sessions, queries, relational identifiers and model persistence semantics. PostgreSQL preserves the current architecture with minimal changes and is therefore the more professional deployment choice.

Production runtime checks

- Production rejects SQLite and requires a persistent database URL.
- Workspace isolation requires a configured signing secret.
- Production rejects local or Docker validation and requires static mode on the public free host.
- Allowed hosts and CORS origins are environment-configured rather than wildcarded.

CHAPTER 17

API Reference

Method	Route	Purpose
GET	/health	Safe service, AI and runtime state
GET	/dashboard	Workspace metrics
POST	/documents/upload	Upload, redact and discover systems
GET	/documents	List workspace documents
GET	/inventory	Query workspace inventory
DELETE	/inventory	Clear current workspace inventory
GET	/inventory/export/json	Export inventory JSON
GET	/inventory/export/csv	Export inventory CSV
POST	/use-cases	Create use case
GET	/use-cases	List use cases
POST	/use-cases/discover	Discover candidate goals
POST	/gap-analysis	Create ad-hoc gap analysis
GET	/gaps/{id}	Generate/retrieve gap report
POST	/generate-connectors	Generate connector package
GET	/artifacts	List generated artifacts
POST	/validate	Validate supplied/generated artifact
GET	/validations	Validation history
GET	/reports	Consolidated report payload
GET	/reports/gaps/{id}.json	Export gap report
POST	/integrations/llm/verify	Verify providers in failover order
POST	/integrations/groq/verify	Verify Groq only
POST	/integrations/gemini/verify	Verify Gemini only

CHAPTER 18

Configuration Reference

Variable	Default	Purpose
DISCOVERY_ENVIRONMENT	development	development / production behavior
DISCOVERY_DATABASE_URL	SQLite local	SQLModel connection; PostgreSQL for production
DISCOVERY_GROQ_API_KEY	unset	Groq provider credential
DISCOVERY_GROQ_MODEL	openai/gpt-oss-120b	Groq model
GEMINI_API_KEY	unset	Google Gemini SDK credential
DISCOVERY_GEMINI_MODEL	gemini-3.5-flash-lite	Gemini model
DISCOVERY_LLM_PROVIDER_ORDER	groq,gemini	provider failover order
DISCOVERY_CORS_ORIGINS	local Vite origins	allowed browser origins
DISCOVERY_ALLOWED_HOSTS	local + Render wildcard	trusted Host headers
DISCOVERY_SESSION_SECRET	unset	workspace cookie signing secret
DISCOVERY_WORKSPACE_ISOLATION	false	anonymous user workspace separation
DISCOVERY_RATE_LIMIT_ENABLED	true	protect expensive endpoints
DISCOVERY_MAX_UPLOAD_BYTES	20 MiB	upload size cap
DISCOVERY_MAX_PERSISTED_TEXT_CHARS	100000	redacted text retention cap
DISCOVERY_RETAIN_UPLOADS	true	retain/delete original uploads
DISCOVERY_VALIDATION_MODE	docker	static / docker / local validation

CHAPTER 19

Testing, CI and Quality Engineering

Automated quality gates

- Pytest backend suite.
- Python compile check.
- ESLint frontend linting.
- Vitest + Testing Library frontend tests.
- TypeScript production compilation.
- Vite production build.
- Docker image builds.
- Docker Compose configuration validation.
- Docker sandbox connector validation in CI.

Continuous integration

GitHub Actions runs backend and frontend jobs in parallel, then builds deployment containers after both pass. The Compose validation uses quiet mode so environment expansion does not echo credentials into logs.

Current integration note

Deployment-hardening regression work

The stable pre-hardening baseline passed the complete local quality gates. The newly added workspace/session middleware changes API test assumptions, so TestClient fixtures must obtain/use the signed workspace cookie (or explicitly disable isolation) before the public-deployment branch is considered release-ready.

CHAPTER 20

Worked Example: Invoice Automation

Input estate

Assume an uploaded architecture document explicitly mentions Salesforce and NetSuite but does not document Stripe as an existing production system.

Use case

Field	Value
Title	Invoice Automation
Business goal	Create invoices automatically after commercial approval
Target systems	Salesforce, NetSuite, Stripe
Frequency	Daily
Criticality	High

Expected analysis

System	Status	Interpretation
Salesforce	Available	Discovered from evidence
NetSuite	Available	Discovered from evidence
Stripe	Missing	No evidence in current inventory

The missing Stripe capability becomes a high-priority integration gap. Discovery Agent can generate a connector scaffold and validation assets, while preserving the fact that the integration is not yet an observed part of the enterprise estate.

CHAPTER 21

Professionalism and Unique Design Choices

What makes the project stronger than a generic AI demo

20. Evidence is first-class data. Findings are not just labels; they carry source context.
21. The system is hybrid rather than model-dependent. External AI failure does not collapse the application.
22. Hallucination control checks model output against source text before persistence.
23. Generated code is treated as untrusted and validated according to environment.
24. Deployment security is explicit: workspace isolation, rate limits, security headers and safe defaults.
25. The UI is an operator console, not a marketing page with decorative AI effects.
26. The project documents its production boundaries instead of claiming enterprise readiness without controls.

Potential signature feature: Trust & Runtime panel

A distinctive future enhancement is a visible Trust & Runtime panel showing the active AI provider, failover order, validation mode, privacy retention mode, workspace isolation and evidence coverage. This would make operational trust state inspectable rather than hidden in configuration.

CHAPTER 22

Limitations and Risk Register

Risk / limitation	Impact	Mitigation
Pattern-only redaction	Sensitive names may remain	managed DLP/NLP redaction for sensitive deployments
In-memory rate limits	not shared across instances	Redis or gateway-level rate limiting
Heuristic confidence	not calibrated probability	labelled benchmark and calibration
Generic connector scaffolds	vendor API mismatch possible	contract tests and vendor adapters
Free-host cold starts	slow first request	paid always-on compute for production
No full RBAC yet	anonymous isolation is not enterprise identity	Supabase/Auth0/enterprise SSO + roles
No migrations yet	schema upgrades harder	Alembic
Static hosted validation	cannot prove runtime behavior	dedicated remote sandbox worker
AI free-tier quotas	requests can fail	provider routing + deterministic fallback

CHAPTER 23

Roadmap

Priority	Work
P0 - Release hardening	Update API integration tests for signed workspaces; finalize Render/Supabase configuration; complete same-origin frontend deployment.
P1 - Identity	Add Supabase Auth or enterprise OIDC, roles and workspace ownership.
P1 - Database lifecycle	Alembic migrations, indexes, retention jobs and backup policy.
P1 - Observability	Structured JSON logs, request correlation, error monitoring, metrics and traces.
P2 - Sandbox service	Dedicated isolated worker for safe runtime connector testing.
P2 - Evaluation	Labelled extraction dataset and calibrated precision/recall/confidence metrics.
P2 - Connector adapters	Vendor-specific API contracts, auth flows and integration tests.
P3 - Enterprise controls	SSO, audit trail, policy engine, secrets manager, data retention and governance.

CHAPTER 24

Local Development and Operations

Prerequisites

- Python 3.12
- Node.js 22+ and npm
- Docker Desktop/Engine for secure local connector validation

First-time setup

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# Linux/macOS: source .venv/bin/activate
python -m pip install -r requirements.txt
cd frontend
npm ci
cd ..
```

Run

```
python scripts/dev.py --check
python scripts/dev.py
```

Verify

```
python scripts/verify.py
```

Local endpoints

Service	URL
React UI	http://127.0.0.1:5173
FastAPI	http://127.0.0.1:8000
Health	http://127.0.0.1:8000/health
OpenAPI	http://127.0.0.1:8000/docs

CHAPTER 25

Deployment Checklist

Check	Release status
Secrets rotated and not committed	Required
Production PostgreSQL URL configured	Required
DISCOVERY_ENVIRONMENT=production	Required
Workspace isolation enabled	Required
Strong session signing secret configured	Required
Validation mode set to static on free public host	Required
CORS set to exact frontend origin	Required
Allowed hosts set to deployed hostnames	Required
Original upload retention disabled unless explicitly needed	Recommended
AI provider keys configured server-side only	Required
API and frontend build from clean commit	Required
Regression tests updated and green	Required
Rate limits verified	Required
Security headers verified in deployed environment	Required
README deployment section matches actual stack	Required

CHAPTER 26

Viva and Interview Questions

Q: Why did you use a hybrid deterministic + LLM architecture?

A: To avoid making core behavior dependent on an external model. Deterministic logic gives predictable fallback; LLMs improve interpretation and generation when available.

Q: How do you reduce hallucination?

A: A model-discovered system is accepted only when its name or meaningful terms are grounded in the source chunk. Evidence is persisted with the finding.

Q: Why SQLAlchemy?

A: It combines Pydantic-style typing with SQLAlchemy persistence and gives a clean path from SQLite development to PostgreSQL deployment.

Q: Why not MongoDB?

A: The domain is already modeled relationally; PostgreSQL preserves SQLAlchemy and avoids a persistence rewrite.

Q: Why is Docker validation important?

A: Generated code is untrusted. Docker can remove network access, bound resources and prevent host-level execution.

Q: Why static validation in public hosting?

A: Free hosts do not provide a safe nested container runtime. Static validation avoids executing generated code inside the API service.

Q: What is workspace isolation?

A: A signed anonymous identifier scopes all database operations so different public users do not see one another's inventory or artifacts.

Q: How do you protect free AI quota?

A: Provider failover, deterministic fallback and rate limits on expensive endpoints.

Q: What is the most enterprise-specific aspect?

A: Evidence/provenance, explicit trust boundaries, secure connector handling, deployment/runtime controls and honest production limitations.

Q: What would you add for real production?

A: RBAC/SSO, migrations, managed object storage, shared rate limiting, observability, calibrated evaluation and a dedicated sandbox worker.

CHAPTER 27

Glossary

Term	Meaning
API	Application Programming Interface; contract for program-to-program communication.
CORS	Browser policy controlling which origins may read API responses.
Evidence grounding	Requirement that an extracted fact be traceable to source content.
Gap analysis	Comparison between required capabilities/systems and the existing estate.
HMAC	Keyed hash used to sign workspace identifiers so clients cannot forge them.
LLM	Large Language Model.
ORM	Object-Relational Mapper; maps program objects to database rows.
RBAC	Role-Based Access Control.
Static validation	Syntax/security checks without executing generated code.
Sandbox	Isolated environment used to reduce risk when executing untrusted code.
SQLModel	Python library combining Pydantic-style models with SQLAlchemy.
Workspace	Logical data boundary isolating one user/session from another.
Provenance	Metadata describing where a finding came from.
Fail closed	Security behavior where missing protection causes failure rather than unsafe fallback.

CHAPTER 28

Repository Map and File Responsibilities

Path	Responsibility
backend/main.py	FastAPI routes, orchestration, persistence and middleware integration
backend/settings.py	environment-aware configuration and runtime safety checks
backend/models.py	SQLModel persistence schema
backend/core/ingestor.py	file parsing and chunking
backend/core/redactor.py	pattern redaction
backend/core/extractor.py	system discovery and evidence logic
backend/core/discovery_catalog.py	known-system catalog
backend/core/provider_router.py	AI provider failover
backend/core/llm.py	Groq adapter
backend/core/gemini_provider.py	Gemini adapter
backend/core/mapping_engine.py	gap analysis
backend/core/code_gen.py	connector generation
backend/core/sandbox.py	static/local/Docker validation
backend/core/security.py	workspace cookie, rate limits, security headers
frontend/src/App.tsx	frontend coordinator/state
frontend/src/pages/*	page-level UI
frontend/src/services/api.ts	typed HTTP client
frontend/src/types/api.ts	TypeScript API contracts
scripts/dev.py	cross-platform development launcher
scripts/verify.py	quality-gate runner
.github/workflows/ci.yml	CI pipeline

CHAPTER 29

Conclusion

Discovery Agent demonstrates more than document extraction or code generation. Its engineering value comes from combining evidence-based AI behavior, integration architecture, secure execution boundaries, typed persistence, provider resilience, a professional operator interface and deployment-aware security controls into one coherent workflow.

The project is strongest when presented as a production-oriented prototype with explicit trust mechanisms and a clear path to enterprise hardening. The next milestone is to complete the hosted deployment branch, update the regression tests for workspace isolation, connect managed PostgreSQL, and publish a stable free demo that preserves the same evidence-first philosophy as the local system.

One-sentence project explanation

Discovery Agent turns messy enterprise documentation into a source-grounded application inventory, identifies what integrations are missing for a business goal, generates connector scaffolds, and validates them using environment-appropriate safety controls.

CHAPTER 30

Appendix A - Suggested Production Environment

```
DISCOVERY_ENVIRONMENT=production
DISCOVERY_DATABASE_URL=postgresql+psycopg://<managed-postgres-url>
DISCOVERY_WORKSPACE_ISOLATION=true
DISCOVERY_SESSION_SECRET=<strong-random-secret>
DISCOVERY_VALIDATION_MODE=static
DISCOVERY_RETAIN_UPLOADS=false
DISCOVERY_RATE_LIMIT_ENABLED=true
DISCOVERY_CORS_ORIGINS=https://<frontend-host>
DISCOVERY_ALLOWED_HOSTS=<api-host>
DISCOVERY_LLM_PROVIDER_ORDER=gemini,groq
GEMINI_API_KEY=<server-side-secret>
DISCOVERY_GROQ_API_KEY=<server-side-secret>
```

Credential handling

Never place real database passwords or AI keys in README files, frontend VITE_* variables, source code, screenshots, issue trackers or chat transcripts. Rotate any credential that has been exposed.